

CS 39006: Computer Networks Laboratory

Non-Blocking I/O

Department of Computer Science
and Engineering



INDIAN INSTITUTE OF TECHNOLOGY
KHARAGPUR



Abhijnan Chakraborty
abhijnan@cse.iitkgp.ac.in

Sandip Chakraborty
sandipc@cse.iitkgp.ac.in

Which Socket Calls Block?

A **blocking call** suspends your thread until the operation completes. Your program is stuck – it can't do anything else while it waits.

`accept()`

Waits until a client connects

`read()` / `recv()`

Waits until data arrives on the socket

`write()` / `send()`

Waits if the kernel send buffer is full

`connect()`

Waits for the TCP 3-way handshake to finish

Key insight: By default, every new socket in C is blocking. The kernel puts your process to sleep until the I/O event occurs.

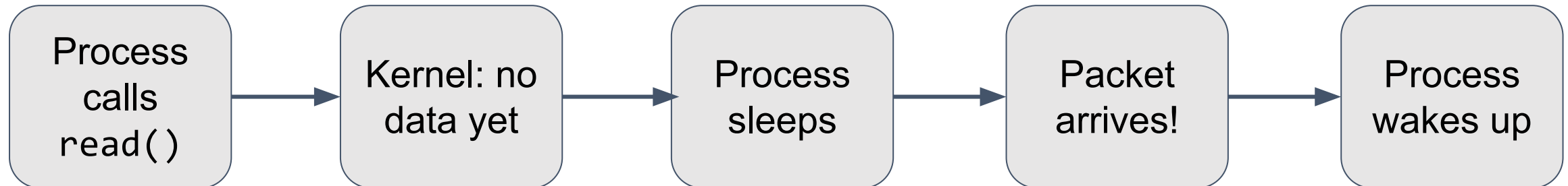
Why Do They Block?

// Your code calls:

```
n = read(sockfd, buf, sizeof(buf));
```

// Inside the kernel:

1. Check socket receive buffer
2. Buffer empty? → put process to SLEEP
3. Data arrives from network → wake process up
4. Copy data to user buffer, return byte count

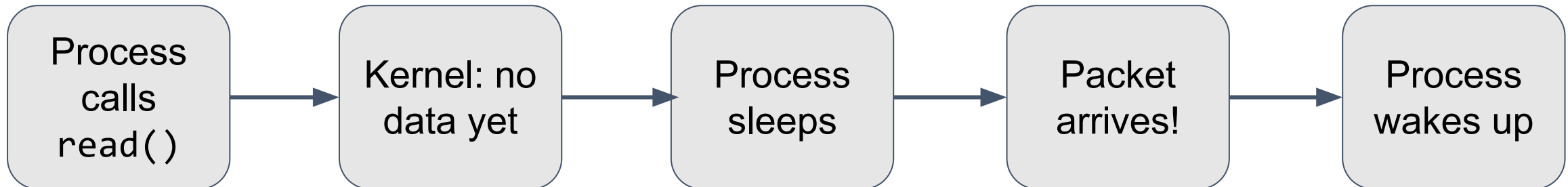


Why Do They Block?

```
// Your code calls:  
n = read(sockfd, buf, sizeof(buf));  
// Inside the kernel:
```

The same pattern applies to `accept()` (waiting for a connection), `connect()` (waiting for handshake), and `write()` (waiting for buffer space).

4. Copy data to user buffer, return byte count



The Problem with Blocking

Imagine a chat server handling multiple clients with blocking I/O

```
// Naive blocking server
while(1) {
    // ⚠ Stuck here until Client A sends data
    n = read(client_a_fd, buf, SIZE);
    // Client B's message? Client C connecting?
    // TOO BAD – we're sleeping on client_a_fd!
}
```

The Problem with Blocking

Imagine a chat server handling multiple clients with blocking I/O



Can't serve multiple clients

Blocked on one socket,
deaf to all others



Wasted time

CPU sits idle while
waiting for network events



Deadlocks possible

Both ends waiting on
each other's read()

}

Wait – can't we just fork()?

Yes! One process per client works, but:



fork() per client

Works, but processes are heavy – memory overhead, IPC complexity, hard to share state between clients



Thread per client

Lighter than fork, but still costly at scale – stack memory, context switches, synchronization bugs



Non-blocking + multiplex

One thread handles ALL clients – no overhead per connection. This is what we'll learn next!

`select()` can also lie!

- Works for classroom assignments, but the real world is much harsher!
- It tells you a fd is "ready," but that doesn't guarantee the subsequent `read()` or `accept()` won't block.
 - For example, `select()` might report a listening socket as readable, but by the time you call `accept()`, the client has already disconnected (TCP RST) – and `accept()` blocks.
 - Similarly, the kernel might report readability based on a packet that later fails a checksum, so `read()` would block.
- `write()` after `select()` reports writable can still block if you try to write more bytes than the available buffer space.
 - `select()` says *"you can write something,"* not *"you can write your entire 10KB message."*

When & Why Go Non-Blocking?

Any time your program must handle multiple I/O sources or stay responsive while waiting:

Web Servers

Thousands of concurrent connections
– can't dedicate a thread to each one

Chat / Messaging

Must read from any client that has data
and broadcast immediately

Game Servers

Must process input from many players
AND run a game loop at 60+ ticks/sec

Proxy / Load Balancer

Relay data between upstream and
downstream – both sides are sockets

Rule of thumb: If you have more than one file descriptor to manage, or you need to do work between I/O events, you need non-blocking I/O.

Option 1: `fcntl()` + `O_NONBLOCK`

Make individual calls return immediately

```
// Set socket to non-blocking mode
int flags = fcntl(sockfd, F_GETFL, 0);
fcntl(sockfd, F_SETFL, flags | O_NONBLOCK);
// Now read() returns immediately:
n = read(sockfd, buf, SIZE);
if (n == -1 && errno == EAGAIN) {
    // No data yet – not an error! Go do other work.
}
```

`fcntl() + O_NONBLOCK`

Make individual calls return immediately

✓ Pros

Simple concept. Call returns instantly — you decide when to retry.

✗ Cons

Busy-waiting! Looping over all fds wastes CPU. You need to poll yourself.

```
n = read(sockfd, buf, SIZE);  
if (n == -1 && errno == EAGAIN) {
```

Rarely used alone. Almost always combined with a multiplexing mechanism (next slides).

select() with Non-Blocking I/O

The classic – monitor many fds at once

```
fd_set readfds;
// Make sockets non-blocking first
fcntl(client1_fd, F_SETFL, fcntl(client1_fd, F_GETFL) | O_NONBLOCK);
FD_ZERO(&readfds);
FD_SET(listen_fd, &readfds);
FD_SET(client1_fd, &readfds);
// Wait until ANY fd is ready (efficient!)
select(maxfd + 1, &readfds, NULL, NULL, &timeout);
if (FD_ISSET(client1_fd, &readfds)) {
    n = read(client1_fd, buf, SIZE);
    if (n == -1 && errno == EAGAIN) // rare spurious readiness – safe!
        continue;
}
```

Option 2: SOCK_NONBLOCK at creation

```
// Set non-blocking at creation (Linux 2.6.27+)
sockfd = socket(AF_INET,
                SOCK_STREAM | SOCK_NONBLOCK, 0);
// One call, atomic, no race condition
```

Don't forget accepted sockets! Even if your listening socket is non-blocking, each new fd from `accept()` inherits the default (blocking) mode. You must set `O_NONBLOCK` on it too; or use `accept4()` with `SOCK_NONBLOCK`.

What Changes When the Socket Is Non-Blocking?

Call	Blocking behavior	Non-blocking behavior
<code>accept()</code>	Sleeps until a client connects	Returns EAGAIN if no pending connections
<code>read()</code>	Sleeps until data arrives	Returns EAGAIN if buffer is empty
<code>write()</code>	Sleeps if send buffer is full	Returns EAGAIN if buffer is full; may write fewer bytes than requested (partial write)
<code>connect()</code>	Sleeps until handshake completes	Returns immediately with EINPROGRESS; use select/poll to detect completion

The trade-off: Non-blocking sockets never stall your program, but you must handle EAGAIN, partial reads/writes, and track per-connection state yourself. This complexity is the price of scalability.

Per-Call Non-Blocking: MSG_DONTWAIT

Sometimes you don't want the entire socket to be non-blocking – you just want one specific call to return immediately. The MSG_DONTWAIT flag does exactly this.

```
// Socket is still blocking, but THIS
// call returns immediately
n = recv(sockfd, buf, SIZE, MSG_DONTWAIT);
if (n == -1 && errno == EAGAIN)
    // No data yet – socket stays blocking
    // for all other calls
```